Università di Catania

DIPARTIMENTO DI INGEGNERIA
ELETTRICA ELETTRONICA E INFORMATICA

CORSO DI LAUREA MAGISTRALE IN INGEGNERIA INFORMATICA

Music Copilot AI

Studenti:
Ficarra Filippo 1000034832
Guerrera Mario Anthony 1000035663

Anno Accademico 2025/2026

Sommario

1. Descrizione progetto 1
   1.1. Tecnologie utilizzate 1
2. Architettura e Descrizione del Workflow 2
   2.1. Gestione dello stato 2
   2.2. Workflow 3
   2.3. Planner 5
   2.4. Researcher 6
   2.5. Writer 7
3. La componente semantica: Il Knowledge graph editoriale 9
   3.1. Sincronizzazione Preventiva del Canale Storico ("get_covered_topics_and_recency") 10
   3.2. Matching Semantico (Vettorializzazione in RAM) 10
   3.3. Il Nodo di Consolidamento ("update_kg_node") 10
   3.4. Le Query Cypher Transazionali di Scrittura ("add_approved_post") 11
4. K-RAG Locale 12
   4.1. La Pipeline di Ingestion e Indicizzazione (populate_pdf.py) 12
   4.2. Il Database Vettoriale e lo Spazio Latente (vector_manager.py) 13
      4.2.1. Codifica Vettoriale (Dense Vector Embedding) 13
      4.2.2. Persistenza d'Archivio (On-Disk Persistent Collection) 13
   4.3. L'Integrazione Bidirezionale K-RAG (rag_tool.py) 13
      4.3.1. Espansione della Query Guidata dal Grafo 13
      4.3.2. Filtro di Densità Lessicale 13
      4.3.3. Ricerca per Similarità sul Coseno 14
5. Tool Utilizzati 15
6. Fine-tuning 16
   6.1. Definizione del problema 16
   6.2. Dataset utilizzato 17
      6.2.1. Risoluzione dei bias 17
   6.3. Metriche di valutazione 18
   6.4. Baseline 19
   6.5. Scelta del modello 20
   6.6. Prima strategia di Fine-Tuning 20
      6.6.1. Ricerca Iper Parametri per LoRA 21

6.6.2. Fine-tuning con LoRA 22
6.6.3. Testing con LoRA 22
6.6.4. Error Analysis 23
6.7. Seconda strategia di Fine-tuning 23
6.7.1. Scelta degli iperparametri per il Full fine-tuning 24
6.7.2. Fine-tuning con FFT 24
6.7.3. Testing con FTT 24
6.8. Confronto con Baseline 25
7. Valutazione 26
7.1. Analisi Qualitativa dei Post Generati 26
7.2. Valutazione della Qualità delle Fonti e del Grounding (Fondatezza) 27
7.3. Metriche di Osservabilità 28
7.4. Identificazione dei Casi di Fallimento 29

1. Descrizione progetto

Il sistema multi-agentico sviluppato prevede la pianificazione, creazione e pubblicazione di post relativi al dominio musicale. Possono essere realizzate 4 diverse categorie di post:

1. **Eventi Imminenti sul Territorio:** si tratta di post sui prossimi festival, concerti ed eventi musicali. Per questa categoria, il modello è guidato, tramite prompt, a pianificare post e ad effettuare ricerche riguardo eventi svolti in Sicilia, e a partire dall'estate 2026.
2. **Guide Pratiche e Didattiche:** Guide pratiche, tutorial tecnici o didattici. Esempi concreti di post generati includono guide sulla regolazione dell'audio, applicazione di tecniche di compressione dinamica della voce, o tutorial sul cambio di corde per chitarra o per suonare strumenti vari.
3. **Recensioni di Prodotti e Analisi di Cantanti:** Recensioni di prodotti musicali, o analisi critiche e storiche della discografia e della carriera di un artista.
4. **Progressi e Novità del Settore:** Progressi tecnologici, innovazioni scientifiche e novità del dominio audio/musicale. Esempi di post generati riguardano soprattutto la rivoluzione dell'ambito musicale, dovuta all'applicazione dell'Intelligenza artificiale.

L'obiettivo strategico è affermare il blog come una fonte di informazione autorevole, rigorosa e scientificamente fondata. Per fare ciò, l'architettura è stata ingegnerizzata per supportare la pianificazione e scrittura di 3 post ad esecuzione, e la pubblicazione automatizzata e coerente di un post a cadenza regolare giornaliera.

1.1. Tecnologie utilizzate

- **LangGraph:** È adatto perché permette di realizzare workflow complessi, gestendo in modo deterministico lo stato condiviso (AgentState). Si integra coordinando il passaggio dello stato tra i nodi e abilitando i punti di interruzione asincroni, in modo da favorire l'interazione con l'utente (Human-in-the-loop).
- **Neo4j:** Scelta per la sua natura nativamente relazionale e basata su query Cypher, ideale per mappare entità interconnesse come Post, Topic, Claim e Fonti. Si integra come memoria semantica a lungo termine del blog.
- **Gemini 2.5 Flash:** Modello scelto per la sua eccezionale velocità di elaborazione e basso costo di token. Si integra con successo nei vari scenari di pianificazione, ricerca e scrittura. Infatti, viene configurata a temperatura zero nel Researcher per guidare la logica formale dei tool, e a temperatura 0.7 nel Writer per garantire fluidità e stile editoriale nella stesura.
    - NOTA: Nonostante il basso consumo dei token per il piano gratuito, dal momento che è stato scelto un numero di 3 articoli prodotti per esecuzione, è stata implementata, di conseguenza, una funzione di rotazione dell'utilizzo di 2 chiavi API (all'interno del file "config.py"), permettendo la stesura di 3 post senza saturare le richieste giornaliere.
- **ChromaDB:** È un database vettoriale per la conoscenza non strutturata (PDF su manuali tecnici e sulle storie di cantanti famosi) e il recupero K-RAG, adatto per la velocità di indicizzazione on-disk e la flessibilità e semplicità di configurazione.

- **Gemini-Embedding-2:** Utilizzato come algoritmo di rappresentazione semantica densa. È adatto perché mappa i concetti testuali in coordinate geometriche multidimensionali (spazio latente), catturando il vero significato dei termini. Si integra sia per indicizzare e ricercare i chunk in ChromaDB, sia per eseguire il calcolo in RAM della similarità del coseno tra i vecchi topic di Neo4j.
- **Tavily Search (Motore di Ricerca Web Ottimizzato):** Un servizio di esplorazione web ingegnerizzato appositamente per agenti AI, in grado di bypassare il rumore di una ricerca Google standard, restituendo snippet puliti e URL tracciabili.

2. Architettura e Descrizione del Workflow

L'intera orchestrazione della pipeline è stata implementata tramite LangGraph, ideale per implementare pattern di ragionamento avanzati come il ReAct.

2.1. Gestione dello stato

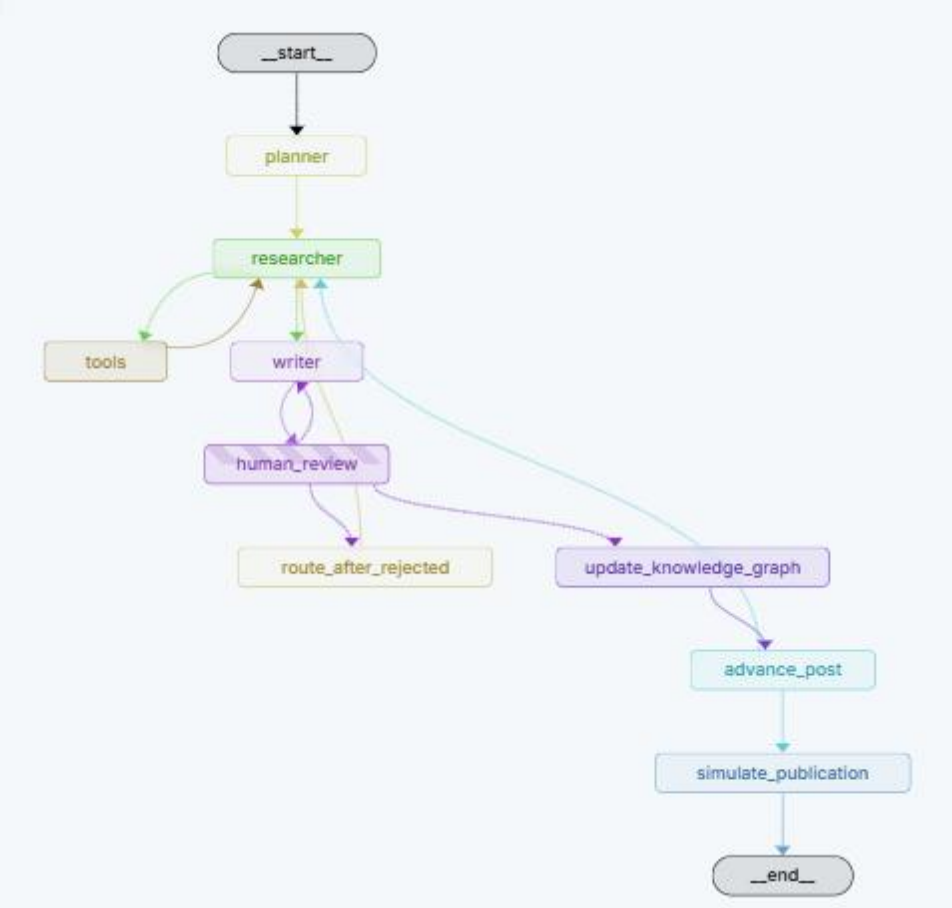
Il cuore del sistema è rappresentato dallo stato condiviso "AgentState", definito all'interno del modulo "state.py". Questo oggetto funge da memoria centralizzata a breve termine dell'agente, memorizzando le informazioni necessarie per il passaggio del contesto tra i vari nodi del workflow. Lo stato è formato dai seguenti campi:

- "proposed_post": Sequenza di post pianificata. Include, per ogni post proposto, il topic affrontato, il tipo di post (tra i 4 visti in precedenza) e la giustificazione sul motivo di scelta del topic (basata sul contenuto del Knowledge Graph).
- "current_post_index": indice del post attualmente in fase di scrittura.
- "current_target_topic": Rappresenta un elemento specifico di "proposed_post". Contiene il topic, il tipo e la giustificazione sulla scelta del topic, relativi al post attualmente in fase di scrittura.
- "context_knowledge_graph": Se nel knowledge graph è presente un topic correlato al topic attuale proposto, questo campo contiene un flag true/false, il nome del topic, il contenuto del post associato al topic e i claim estratti dal post.
- "recovered_documents_RAG": Viene popolato in seguito alla ricerca sul ChromaDB con i due documenti maggiormente pertinenti alla query di ricerca (decisa dal modello). Per ogni elemento memorizzato, è presente il contenuto del chunk, lo score realizzato durante il recupero K-RAG e la provenienza di tale contenuto (formata dal titolo del file.pdf e la pagina precisa).
- "messages": è importante per implementare il pattern ReAct in fase di ricerca (Thought->Action->Observation).
    - NOTA: È stato scelto di includere, in questo campo, sia la traccia di ragionamento formulata, sia gli output dei tool (ciascun output è un messaggio contenente id, nome del tool, e risultati ottenuti)
- "researcher_status": può essere "IN_PROGRESS" o "COMPLETE", indicando se la ricerca è terminata.
- "generated_draft": articolo finale generato.

- “feedback”: viene usato per implementare il meccanismo di Human in the loop. Infatti, questo campo può essere popolato dall’utente con una stringa “approved”, “rejected” o “modified”.
- “notes”: Se l’utente desidera modificare l’articolo (opzione “modified”) può inserire in questo campo una stringa che verrà catturata per la riscrittura dell’articolo.
- “approved_post”: Lista contenente i post approvati dall’utente in una determinata esecuzione.
- “published_post”: Contiene il post appena pubblicato sul database Neo4J.
- “post_to_publish”: Contiene tutti i post, in coda, approvati da pubblicare sul database.
- “publication_schedule”: Se i post da pubblicare non dovessero essere ordinati, verrebbero ordinati in questo campo in ordine crescente di data, per una futura pubblicazione.
- “publication_summary”: indica se un contenuto è stato pubblicato e quanti ne rimangono in coda.

2.2. Workflow

L'architettura è composta da diversi nodi, ciascuno deputato a svolgere un compito specifico all'interno del dominio editoriale.



1. **planner:** È il nodo d'ingresso del workflow. Interroga il database Neo4j (e analizza il “publication_state.json”, nel quale sono contenuti i post in coda approvati dall’utente) per mappare lo storico dei contenuti già trattati. Identifica i gap semantici

e genera un array JSON contenente una roadmap di esattamente 3 post nuovi e diversificati, impostando l'indice di avanzamento (“current_post_index”) a 0.

2. **researcher:** Opera secondo il pattern ReAct, formulando un THOUGHT dovuto alle osservazioni accumulate dai tool, ed eseguendo una ACTION di conseguenza. È l'unico nodo deputato alla selezione dinamica dei tool.
3. **tools:** Un’istanza predefinita di ToolNode che incapsula ed esegue i 5 strumenti a disposizione del sistema (“web_search_tool”, “rag_retrieval_tool”, “query_knowledge_graph_tool”, “source_credibility_tool”, “web_content_moderator_tool”). Restituisce l'output (OBSERVATION) come messaggio all’interno del campo “messages” dello stato, il quale viene poi prelevato dal researcher.
4. **writer:** Prende il controllo quando la ricerca è conclusa. Sfrutta il contesto del Knowledge Graph, i testi estratti dal K-RAG e le ricerche web effettuate per redigere la bozza finale, inserendo le citazioni e processando i punteggi di affidabilità delle fonti calcolati dal “source_credibility_tool”.
5. **human_review:** Un nodo passivo progettato per implementare il meccanismo di Human-in-the-loop. Sfrutta un punto di interruzione per congelare il grafo e attendere il verdetto dell'utente.
6. **route_after_rejected:** Se l'utente rifiuta la bozza, questo nodo azzera i campi dello stato, in particolare il “context_knowledge_graph”, il “recovered_documents_RAG” e i “messagges” non più validi, prima di rispedire il controllo al researcher per una nuova esplorazione, prevenendo allucinazioni.
7. **update_knowledge_graph:** Viene eseguito solo dopo l'approvazione umana. Estrae i fatti chiave (claim) dall'articolo e inserisce in coda il post all'interno del “publication_state.json”.
8. **advance_post:** Incrementa l'indice della roadmap (“current_post_index”) per scorrere la sequenza dei post prefissati (“proposed_post”) e permettere la scrittura del prossimo.
9. **simulate_publication:** Nodo finale del workflow. Verifica se è passato almeno un giorno (soglia scelta dal team) dall'ultima pubblicazione. Se la condizione è verificata estrae il post più vecchio dal ”publication_state.json” e lo inserisce in modo permanente all'interno del database semantico Neo4j, aggiornando le relazioni del grafo.

Nel workflow, oltre agli archi che collegano in maniera diretta i nodi visti prima, sono presenti 3 archi condizionali, che dipendono dai risultati di 3 ulteriori funzioni:

1. **route_after_research:** Dopo l'esecuzione del researcher, esamina l’ultimo messaggio della cronologia “messages”. Se l'LLM ha emesso una richiesta di tool (tool_calls), il flusso devia verso il nodo tools. Se, recuperando il campo “research_status”, questo risulta “COMPLETE” il flusso si sblocca indirizzandosi al nodo writer.
   - NOTA: Il researcher può settare il campo “research_status” a “COMPLETE” se, in seguito al THOUGHT, si rende conto di avere tutte le informazioni necessarie, oppure nel caso in cui dovesse raggiungere un numero massimo di chiamate (impostato a 9)

ai tool. Ques'ultima scelta è stata effettuata per evitare la saturazione dei token del modello "Gemini 2.5 flash".

2. **route_after_human_review:** Dopo il nodo "human_review", esamina il campo "feedback" dello stato. Se l'articolo è approvato ("approved"), si passa alla scrittura incrementale sul "publication_state.json" (tramite la funzione "update_knowledge_graph" del file "kg_manager.py"); se viene rifiutato ("rejected"), il controllo passa al route_after_rejected, che azzera tutti i campi dello stato settati durante la ricerca, per poi riavviare il researcher. Infine, se lo stato è "modified", il controllo passa al writer.
3. **route_for_next_post:** Dopo che un post è stato convalidato, l'arco verifica se ci sono ancora post da elaborare, tramite un controllo sul "current_post_index". Eventualmente, il grafo pulisce i messaggi e fa ripartire il flusso dal researcher sul nuovo topic. In caso contrario, il flusso viene indirizzato al simulate_publication (per pubblicare un post del "publication_state.json" sul database Neo4j).

2.3. Planner

Il nodo planner (implementato nel file "planner.py" tramite la funzione "plan_topics_node") funge da Direttore Editoriale dell'intera architettura. Il suo compito primario consiste nel mappare lo stato pregresso della conoscenza del blog per definire la linea strategica editoriale in totale autonomia, identificando i gap informativi ed escludendo contenuti ridondanti.

L'algoritmo di pianificazione esegue in modo sequenziale le seguenti operazioni:

1. **Analisi dei Gap su Neo4j:** Il nodo istanzia la classe "EditorialKGManager" per effettuare una query transazionale sul database a grafo. Estrae così l'elenco dei topic già trattati e consolidati, garantendo il rilascio immediato della connessione (kg_manager.close()) all'interno di un blocco finally per prevenire memory leaks. La stessa estrazione viene effettuata per i topic degli articoli in coda, all'interno del "publication_state.json".

2. **Generazione Strategica della Roadmap:** Lo storico viene serializzato in formato JSON e iniettato nel prompt del modello Gemini 2.5 Flash. Configurato con una temperatura conservativa di 0.3, il modello esegue una gap analysis del dominio e pianifica una sequenza strutturata di esattamente 3 post completamente nuovi, assegnando a ciascuno una categoria (Product review, How to, Domain advances, Upcoming events) e una specifica giustificazione editoriale.

3. **Parsing Robusto e Fault Tolerance (Failsafe):** Poiché l'LLM restituisce una stringa, il codice applica un'espressione regolamentare (re.search) per isolare chirurgicamente l'array JSON puro. Qualora l'invocazione API remota o il parsing dovessero fallire (es. per un errore HTTP 503 o 429), il sistema attiva una funzione di ripiego build_fallback_plan(), la quale inietta una roadmap predefinita, salvaguardando la continuità operativa del grafo (Fault Tolerance).

    - NOTA: la roadmap dovrebbe essere aggiornata periodicamente, per evitare che vengano proposti topic già trattati in precedenza.

4. **Inizializzazione dello Stato Condiviso (AgentState):** Una volta convalidato il piano, il modulo estrae il primo post della sequenza e lo assegna al campo “current_target_topic” per indirizzare il successivo nodo di ricerca. Infine, restituisce un dizionario che configura i contatori di avanzamento e resetta la lista “messages” a vuoto, garantendo che l'agente Researcher inizi il proprio ciclo ReAct senza scorie o bias cognitivi pregressi.

2.4. Researcher

Il nodo researcher (implementato nel file “researcher.py” tramite la funzione “react_research_node”) implementa il pattern ReAct (Reasoning and Acting). Questo paradigma combina le capacità di destrutturazione logica del problema (Reasoning) con l'interazione dinamica con l'ambiente circostante tramite invocazione di tool (Acting).

Il ciclo opera come una macchina a stati ricorsiva strutturata secondo la sequenza Thought->Action->Observation:

1. **Thought:** L'LLM analizza l'obiettivo editoriale e la storia pregressa dei messaggi (attraverso il campo “messages”) per valutare lo stato attuale delle proprie informazioni.
2. **Action:** Sulla base del proprio pensiero, il modello decide in totale autonomia quale tool invocare (ad esempio, effettuare una ricerca web o interrogare la base semantica) formulando i parametri di input corretti (come le query).
3. **Observation:** il tool viene eseguito e restituisce un'osservazione (un ToolMessage). Il researcher acquisisce questo nuovo dato e ricomincia il ciclo, valutando se le informazioni ottenute sono esaustive o se necessitano di ulteriori passaggi.

Per garantire la massima linearità di esecuzione nel ciclo ReAct, il modello adottato (Gemini 2.5 Flash) viene configurato con una temperatura rigidamente impostata a 0, azzerando la varianza stocastica e costringendo il modello a prendere decisioni puramente logiche, deterministiche e focalizzate sul contesto. I tool disponibili vengono mappati ed esposti dinamicamente tramite la funzione nativa bind_tools (tools_list).

Viene effettuata una rigorosa logica di controllo dello stato della conversazione, progettata per arginare le derive comportamentali tipiche degli agenti autonomi, quali i loop infiniti e il consumo incontrollato di token (Context Window Saturation):

- **Contatori dello Stato dei Messaggi:** Il nodo esegue un'ispezione preventiva sulla cronologia dei messaggi accumulati nello stato condiviso (AgentState), quantificando il numero totale di chiamate ai tool effettuate fino a quel momento (num_tool_calls)
- **Iniezione Dinamica del Guardrail di Credibilità:** Al raggiungimento dell'ottava chiamata ai tool (num_tool_calls == 8), il nodo effettua una scansione a ritroso per verificare se sia già stato eseguito lo strumento “source_credibility_tool”. In caso negativo, per preservare il vincolo di autorevolezza del blog, il codice inietta chirurgicamente un SystemMessage imperativo d'urgenza. Questo messaggio forza l'LLM a raccogliere tutti gli URL accumulati e a convalidarne la credibilità prima di chiudere la sessione, agendo come un vero e proprio guardrail di sicurezza editoriale.

- **Tetto Massimo Anti-Loop:** Qualora il sistema raggiunga le 9 chiamate ai tool complessive, il nodo attiva una chiusura di sicurezza deterministica, restituendo l'aggiornamento "research_status": "COMPLETE". Questo meccanismo interrompe immediatamente il ciclo ReAct forzando il passaggio al nodo Writer, azzerando i rischi di stallo computazionale. La scelta di tale meccanismo è dovuta alla volontà di evitare la saturazione dei token di Gemini 2.5 flash, considerando che in un'esecuzione verranno scritti 3 post.
- **Risoluzione delle Allucinazioni Sintattiche:** Prima di inviare il prompt a Gemini, il codice controlla che l'ultimo messaggio della cronologia locale sia un ToolMessage (has_active_tool_turn). Questa verifica garantisce la corretta alternanza strutturale dei messaggi imposta dal protocollo di Google, prevenendo crash o fallimenti di validazione sintattica nel backend.

Al termine dell'invocazione, il contenuto testuale della risposta viene normalizzato catturando e unendo le eventuali risposte multiframmente generate da Gemini. Il nodo analizza il testo: se viene rilevata la stringa di terminazione "STATUS: COMPLETE", l'agente dichiara conclusa la raccolta dati aggiornando il campo "research_status" (settato a "COMPLETE"). Se l'LLM non invoca alcun tool e omette la stringa di completamento, il codice intercetta l'anomalia e forza l'uscita in sicurezza impostando lo stato su COMPLETE.

Infine, la funzione restituisce l'aggiornamento a LangGraph, delegando al riduttore nativo (add_messages) l' append del messaggio AIMessage appena emesso, lasciando la memoria di lavoro perfettamente predisposta per l'esecuzione del ToolNode o per la stesura finale del testo.

2.5. Writer

Il nodo writer (implementato nel file "writer.py" tramite la funzione "write_post_node") costituisce il modulo di sintesi esecutiva dell'intera architettura. Il suo obiettivo primario consiste nel raccogliere l'intero patrimonio informativo dallo stato accumulato nelle fasi precedenti per redigere la bozza finale dell'articolo, garantendo al contempo il rigido rispetto dei vincoli di ancoraggio ai dati (grounding).

Il modulo opera come un componente deterministico data-driven, il cui input è interamente definito dallo stato condiviso AgentState. Prima di invocare il modello generativo, il codice esegue una scomposizione e un riassemblaggio strutturato del contesto semantico e documentale:

1. **Iniezione del Contesto Storico (KG):** Il nodo esamina l'oggetto relativo al campo "context_knowledge_graph", estratto, a seguito dell'utilizzo del "query_knowledge_graph_tool", dal database semantico Neo4j. Se l'argomento ha un riscontro storico, il codice isola e formatta i claim e i testi pregressi per garantire la coerenza narrativa; in caso contrario, definisce esplicitamente il tema come un gap di copertura informativo, segnalando all'LLM la totale libertà di tracciare nuove linee divulgative.
2. **Aggregazione e Classificazione dei Messaggi ReAct:** Il codice scansiona l'intera lista dei messaggi accumulati. Intercettando esclusivamente i blocchi di tipo

ToolMessage, il sistema smista e categorizza i flussi in base al tool di origine (web_search_tool, rag_retrieval_tool, source_credibility_tool). I dati estratti vengono unificati in una singola lista strutturata ("all_documents") e serializzati in una stringa densa e progressivamente numerata da 1 a n (per distinguere le fonti), predisponendo il contesto ottimale per la Context Window dell'LLM.

3. **Integrazione Human-in-the-Loop:** il modulo verifica la presenza di correzioni scritte dall'utente nel campo "notes" dello stato, qualora l'utente selezioni l'opzione "modified" (che riporta il flusso allo stesso writer). Qualora l'utente abbia inserito note di revisione o modifiche strutturali, il testo viene inserito nel prompt, forzando l'LLM a rimodellare la generazione secondo le direttive umane. Se l'utente non inserisce alcuna nota di modifica (e questo può capitare o perché l'utente ha selezionato l'opzione "modified" senza alcuna nota aggiuntiva, oppure perché il modulo è eseguito prima dell'interazione Human in the loop), viene applicata una stringa di default che indica nessuna correzione da parte dell'utente.

Per consentire una scrittura fluida e ricca dal punto di vista stilistico ed editoriale, Gemini 2.5 Flash viene configurato in questo nodo con una temperatura pari a 0.7.

Una particolare caratteristica del modulo risiede nell'implementazione della funzione utility proprietaria "extract_credibility_score" e della logica di post-processing basata su espressioni regolari, progettate per eliminare i rischi di allucinazione dei riferimenti:

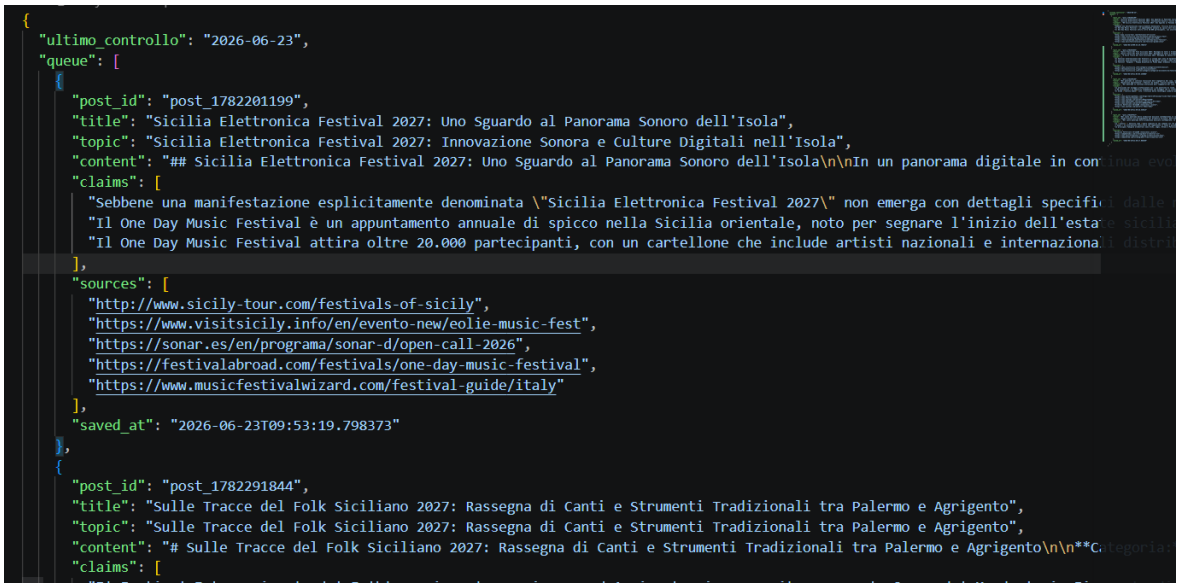
- **Sgancio del Failsafe Interno:** Se un URL analizzato fa riferimento alla manualistica locale indicizzata su ChromaDB, il sistema rileva le stringhe chiave e assegna automaticamente lo status di certificazione interna [Credibilità: Verificata (Interna)], sollevando il codice da calcoli superflui.
- **Estrazione Tramite Regex Sull'Input:** Se la fonte è web, l'utility normalizza l'URL rimuovendo i parametri di tracciamento e i protocolli, trasformando l'intero blocco dei documenti in minuscolo. Viene quindi applicata un'espressione regolare mirata (re.search(rf"{re.escape(url_pattern)}.*?(\d+)\s*/\s*100", ...)) che scansiona i report di credibilità. Questa regex rintraccia l'URL e cattura la prima cifra numerica su base 100 che incontra subito dopo nello snippet, isolandola dal testo. Questo stesso punteggio è stato calcolato in seguito all'applicazione del "source_credibility_tool" e iniettato nella cronologia "messages".
- **Correzione Chirurgica della Bozza:** Una volta emesso l'articolo, un secondo ciclo basato su re.findall individua tutti i link ipertestuali generati dal modello. Qualora Gemini abbia omesso il rating o inserito il valore standard fittizio [Credibilità: Verificata], il codice esegue un rimpiazzo sul posto (.replace()), iniettando la stringa esatta contenente il voto matematico reale (es. [Credibilità: 85/100]) elaborato in precedenza dal "source_credibility_tool".

Questo meccanismo garantisce un grounding biunivoco e matematicamente tracciabile: ogni singola affermazione del post finale è ancorata a una fonte verificata, il cui livello di affidabilità viene marchiato in modo permanente a tutela dell'accuratezza e dell'autorevolezza del blog.

3. La componente semantica: Il Knowledge graph editoriale

La gestione della conoscenza strutturata e dell'evoluzione storica del blog è interamente governata dal modulo “kg_manager.py” (tramite la classe “EditorialKGManager” e la funzione “update_kg_node”) e integrata nella pipeline dallo strumento operativo “query_knowledge_graph_tool”. L'architettura non tratta il database grafico (**Neo4j**) come un semplice archivio statico, ma lo sfrutta attivamente come memoria semantica a lungo termine per abilitare la coerenza narrativa, l'espansione K-RAG e la sincronizzazione preventiva.
La classe “EditorialKGManager” incapsula l'istanza del driver ufficiale di Neo4j e il modello vettoriale gemini-embedding-2.

I post in coda, in attesa di essere pubblicati nel database, vengono memorizzati in un file “publication_state.json”, in attesa che la funzione del nodo del workflow (simulate publication), in seguito ad un controllo sui giorni trascorsi, possa pubblicare il post salvato meno recentemente sul database. La struttura è fil file è mostrata nella seguente immagine.



- NOTA: è stata considerata l’eventualità che il sistema non venga sempre eseguito, anche perché, se così fosse, ogni giorno si avrebbe un post pubblicato contro 3 post generati per esecuzione. Dunque, è stato realizzato, come “ancora di salvezza”, uno script (“run_publication.py”) che implementa lo stesso algoritmo della funzione “simulate_blog_publication_node del nodo simulate_publication. È stata settata un’esecuzione giornaliera in background di questo script. Dunque, in un ipotetico scenario, il sistema potrebbe essere eseguito 2 volte ad inizio settimana, generando una sequenza di post che vengono salvati nel json, e ogni giorno si avrebbe una pubblicazione nel database. La scelta di questa modalità implementativa è dovuta ad una volontà di simulare una pubblicazione vera e propria a cadenza giornaliera di post scritti in maniera autonoma dal sistema.

3.1. Sincronizzazione Preventiva del Canale Storico ("get_covered_topics_and_recency")

- Esegue una query Cypher (MATCH (t:Topic) RETURN t.name) per raccogliere i dati consolidati sul database.
- Ispeziona il file locale "publication_state.json" estraendo la coda dei post approvati dall'utente in attesa di pubblicazione (campo "queue").
- Restituisce la somma unificata di entrambi i canali. In questo modo il Planner visualizza i post in coda come "già coperti", blindando il principio di non-ridondanza ed evitando duplicazioni editoriali.

3.2. Matching Semantico (Vettorializzazione in RAM)

Quando l'agente esegue una query testuale, un matching puramente sintattico (stringa esatta) fallirebbe davanti a sinonimi o flessioni verbali. Il metodo "get_context_for_topic" risolve il problema implementando un motore vettoriale locale direttamente in memoria:

- Estrae tutti i topic testuali registrati su Neo4j.
- Genera l'embedding della query corrente e, in un'unica operazione ad alte prestazioni, calcola i vettori dell'intero storico tramite le API di Google, racchiudendoli in array NumPy.
- Applica in modalità vettoriale la formula matematica della Similarità basata su Coseno:

$$\text{Cosine Similarity} = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|}$$

  Individua l'indice dell'affinità semantica più elevata tramite np.argmax(). Se lo score supera una soglia, il sistema attiva una query Cypher aggregata che si muove lungo i grafi d'archivio (MATCH (p:Post)-[:COVERS]->(t)) raccogliendo in modo atomico (collect(DISTINCT)) sia i claim, sia i testi integrali dei vecchi post per inviarli come contesto di grounding al Writer. Se il punteggio è inferiore, il tema viene marchiato come vero GAP editoriale.
    - NOTA: la scelta della soglia (in questo caso è 0.70) è un trade-off tra evitare di selezionare topic poco correlati al topic attuale, ma allo stesso tempo non riuscire mai a trovare topic correlati, a meno di stretta somiglianza, ma in tal caso non si tratterebbe più di un gap da colmare.

3.3. Il Nodo di Consolidamento ("update_kg_node")

Collocato in coda al punto di interruzione *Human-in-the-loop*, questo nodo scatta non appena l'utente approva l'articolo. Il suo compito è preparare i pacchetti informativi per il futuro aggiornamento incrementale del grafo:

1. **Estrazione e Sanificazione dei Dati:** Genera un identificativo univoco basato su timestamp per evitare collisioni di ID, isola la prima riga del testo per convertirla in titolo pulito ed esegue una scansione Regex globale sui messaggi storici. Se intercetta evidenze da "web_search_tool", estrae gli URL eliminando le scorie di formattazione; se rileva flussi da "rag_retrieval_tool", mappa la traccia della manualistica interna, normalizzando il tutto in un set di fonti univoche.

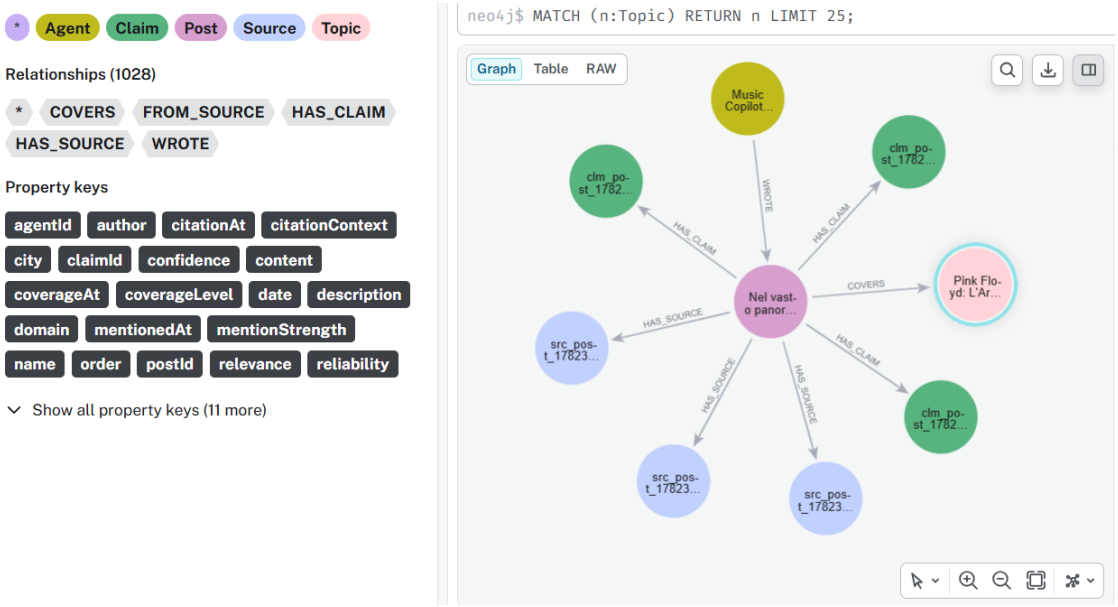
2. **Estrazione delle Asserzioni Chiave (Claim):** Il nodo invoca in modalità isolata Gemini 2.5 Flash impostando la temperatura a 0.1 e un prompt stringente. Il modello estrae da 1 a 3 asserzioni storiche o fatti tecnici descritti nella bozza approvata. Un parsing protettivo via Regex e json.loads converte l'output in un array di stringhe pulito.
3. **Alimentazione del Buffer e Garbage Collection Semantica:** Il post, strutturato con i campi “post_id”, “title”, “topic”, “content”, “claims” e “sources”, viene salvato nel file “publication_state.json”.
   Subito dopo la scrittura sul json, il nodo esegue un'operazione fondamentale per la stabilità del grafo: scansiona la cronologia di lavoro e genera una lista di oggetti RemoveMessage(id=msg.id) per azzerare l'intera memoria dei messaggi ReAct accumulati. Questo svuotamento azzera la Context Window dell'agente, impedendo che i vecchi messaggi del primo post inquinino l'esplorazione dell'articolo successivo della roadmap, garantendo prestazioni ottimali a consumo zero di token. Inoltre, vengono azzerati i campi dello stato settati durante la ricerca, oltre ai campi relativi all’Human-in-the-loop.

3.4. Le Query Cypher Transazionali di Scrittura (“add_approved_post”)

Il metodo “add_approved_post”, richiamato dal nodo simulate_publication, implementa un'architettura relazionale a nodi fortemente tipizzati attraverso query Cypher strategiche:

- **Query Post/Agente:** Sfrutta l'operatore MERGE per accertarsi che il nodo radice dell'agente (:Agent {agentId: "blogger_copilot_2026"}) esista. Crea quindi il nodo :Post, con le varie proprietà, e genera il collegamento direzionale [:WROTE] arricchito con le proprietà di timestamp e conteggio parole.
- **Query Topic/Claim/Source:** Crea, con le varie proprietà, il nodo :Topic e collega il post creato al Topic tramite la relazione [:COVERS]. Successivamente, cicla sulle collezioni creando nodi :Claim e associandoli al post tramite la relazione ([:HAS_CLAIM]) dal post verso il claim, e mappa i nodi :Source (creati a partire dalle fonti estratte) collegando l'origine dei dati sia al post, tramite la relazione [:HAS_SOURCE], dal post alla source, sia ai singoli claim grafici tramite la relazione [:FROM_SOURCE], dal claim alla source.

La seguente figura mostra un esempio di come sono organizzati i nodi nel grafo:



4. K-RAG Locale

Il sistema implementa un modulo di retrieval augmented generation integrata con il Knowledge graph, formalmente definito K-RAG (Knowledge Graph-augmented RAG). Questa componente unisce la conoscenza non strutturata (file PDF incentrati sulle 4 tipologie di post da elaborare) con la conoscenza strutturata di Neo4j, superando i limiti del RAG tradizionale grazie a logiche avanzate di espansione e pre-elaborazione del contesto.

4.1.La Pipeline di Ingestion e Indicizzazione (populate_pdf.py)

Il modulo “populate_pdf.py” governa la pipeline di pre-elaborazione e immissione asincrona dei dati grezzi.

Per evitare la duplicazione dei vettori e l'inquinamento semantico dello spazio latente, il sistema implementa una verifica preventiva. Tramite il metodo rag_manager.vector_store.get(include=["metadatas"]), estrae l'insieme dei metadati storici già indicizzati e isola la chiave "source". L'uso di un Set Python garantisce l'elaborazione esclusiva dei file fisici non ancora presenti nel database vettoriale, rendendo l'operazione interamente idempotente (rieseguibile all'infinito senza alterare lo stato del sistema).

La scomposizione dei testi estratti tramite PyPDFLoader è affidata a una strategia di Heuristic Overlapping Chunking, implementata mediante la classe “RecursiveCharacterTextSplitter”. La configurazione adotta parametri mirati:

- chunk_size=600 caratteri (Dimensione della finestra).
- chunk_overlap=100 caratteri (Fattore di sovrapposizione).

Questa sovrapposizione garantisce che i token di confine mantengano la continuità semantica tra un blocco e il successivo.

Poiché l'API di embedding di Google impone un tetto massimo di richieste al minuto (RPM), l'invio massivo dell'intera collezione provocherebbe un fallimento per “HTTP 429 Too Many Requests”. Per questo motivo, viene inserita un'istruzione

time.sleep(0.75) tra l'indicizzazione di un singolo frammento e il successivo. Questo vincola la pipeline a un massimo di 80 chiamate al minuto, garantendo una stabilità operativa totale e controllata sotto la soglia critica del modulo di embedding.

4.2. Il Database Vettoriale e lo Spazio Latente (vector_manager.py)

La classe “VectorRAGManager” incapsula l'interfaccia con il database vettoriale locale ChromaDB e i modelli di rappresentazione semantica:

4.2.1. Codifica Vettoriale (Dense Vector Embedding)

Il sistema mappa i frammenti di testo in vettori densi a virgola mobile sfruttando il modello di ultima generazione gemini-embedding-2 tramite l'astrazione ufficiale di LangChain. Questo processo trasforma le stringhe di testo in coordinate all'interno di uno spazio geometrico multidimensionale (spazio latente), catturando le relazioni semantiche latenti e il significato profondo dei concetti.

4.2.2. Persistenza d'Archivio (On-Disk Persistent Collection)

Il database Chroma viene inizializzato specificando una persist_directory="./data/chroma_db" e un identificativo di tabella (collection_name="local_knowledge_rag"). Questa configurazione abilita l'On-Disk Indexing, memorizzando in modo permanente le strutture geometriche degli embedding su file system locale. All'avvio dell'applicazione, il grafo esegue un caricamento istantaneo dell'indice precalcolato, azzerando i costi temporali e monetari di re-embedding.

4.3. L'Integrazione Bidirezionale K-RAG (rag_tool.py)

Il tool “rag_retrieval_tool” orchestra una cooperazione strutturata tra Neo4j e ChromaDB attraverso tre passaggi fondamentali:

4.3.1. Espansione della Query Guidata dal Grafo

All'attivazione dello strumento, il sistema non interroga direttamente il Vector DB, ma invia la query del Researcher a Neo4j mediante kg_manager.get_context_for_topic(query). Se il grafo semantico convalida l'esistenza di un'affinità storica pregressa, estrae i nodi Claim associati e attiva la funzione di Query Expansion (Perfezionamento della Query).

4.3.2. Filtro di Densità Lessicale

La stringa dei claim grafici d'archivio viene elaborata dalla funzione “extract_keywords_for_expansion()”, che implementa un algoritmo di sfoltimento lessicale:

- Converte l'input in minuscolo e isola i soli token alfanumerici tramite espressione regolare (re.findall(r'[a-zA-Z0-9àèìòù]+')).

- Sottopone ciascun token a un filtro di esclusione basato su una Blacklist strutturata (stop_words), eliminando articoli, preposizioni e termini ricorrenti del flusso redazionale (es. "guida", "pratica", "manuale").
- Applica un vincolo di lunghezza minima (len(w) >= 4) per escludere frammenti di rumore, e memorizza esclusivamente elementi unici per evitare ridondanze.

I primi 4 token più densi e significativi così estratti vengono concatenati alla query originale (expanded_query = f"{query} {kg_keywords}"), forzando il motore vettoriale a cercare chunk che siano al contempo pertinenti all'obiettivo attuale e in linea con il passato semantico del blog.

  - NOTA: a volte la query del “rag_retrieval_tool” viene espansa anche se nel “query_knowledge_graph_tool” non viene un topic correlato. Questo è dovuto al fatto che è il modello stesso a formulare la query per interrogare il ChromaDB, in base al topic attuale, mentre il “query_knowledge_graph_tool” viene eseguito con il topic stesso. Tale scelta è stata effettuata per poter permettere l’espansione della query anche tramite una strada alternativa, ovvero la capacità di ragionamento e generazione del modello.

4.3.3. Ricerca per Similarità sul Coseno

La query arricchita viene elaborata da ChromaDB eseguendo il metodo similarity_search_with_score(..., k=2). Il database calcola la distanza angolare tra il vettore della query e i vettori dei chunk memorizzati.

Il tool estrae i primi 2 frammenti a massima affinità e ne scompatta il contenuto e i metadati (source e page). Per garantire la massima trasparenza e tracciabilità del dato, il punteggio matematico di distanza viene emesso esplicitamente nel log del Researcher ((Distanza Vettoriale: {doc['score']})).

  - NOTA: Anche se dovessero essere restituiti paragrafi non del tutto inerenti con il topic, ma aventi delle affinità che hanno fatto si avessero un punteggio più elevato di altri, il writer (avendo settato la temperatura del modello a 0.7) agisce in maniera intelligente, includendoli come esempi o paragoni, o per arricchire dei contenuti estendendoli a concetti generali, o non includendoli se nota della forzature nella corrispondenza.

4.3.4. Aggiornamento Combinato dello Stato tramite Command

Il tool conclude la propria esecuzione restituendo un oggetto Command atomico che aggiorna lo stato su due canali paralleli:

- Popola la chiave “recovered_documents_RAG” inserendo un dizionario normalizzato contenente il puro testo del chunk, il file originale e il numero di pagina incrementato (page + 1), poiché il contatore parte da 0.
- Genera un ToolMessage arricchito con i dettagli strutturati della query originale e di quella perfezionata, riproiettando l'osservazione nel campo messages per sbloccare il successivo ciclo di pensiero dell'agente ReAct.

5. Tool Utilizzati

- **query_knowledge_graph_tool**: si connette al database a grafo Neo4j tramite l'istanza di “EditorialKGManager” del “kg_manager.py”. Prende in input la stringa del topic corrente, pulisce la stringa da caratteri speciali ed esegue una query Cypher. Estrae sia i Claim sia i testi integrali di articoli correlati già pubblicati in passato. Permette al writer di evitare contraddizioni con lo storico, o di supportare determinati contenuti grazie ai claim estratti.
- **rag_retrieval_tool**: Implementa il recupero di documentazione di dominio non strutturata (file pdf su manuali, acustica, ingegneria del suono, biografie di cantanti) salvata localmente su ChromaDB tramite la classe “VectorRAGManager”. Il tool genera l’ embedding della query usando il modello gemini-embedding-2 di Google ed estrae i top-k chunk (k è stata posta pari a 2) testuali basandosi sulla similarità del coseno. Il tool implementa una logica di espansione semantica della query guidata dal grafo. Prima di interrogare ChromaDB, estrae i claim storici da Neo4j e li passa alla funzione “_extract_keywords_for_expansion()”. Questa applica una pulizia linguistica (rimozione di *stop-words* generiche e termini colloquiali) per isolare solo i token tecnici unici e densi. Questi token vengono concatenati alla query dell'LLM per massimizzare la precisione del retrieval vettoriale. Infine, Aggiorna lo stato alla chiave “recovered_documents_RAG”, oltre ad inserire i chunck estratti, insieme a titolo, pagina e score, nella cronologia “messages”.
- **web_search_tool**: Consente all'agente di raccogliere evidenze pubbliche, notizie recenti, progressi tecnologici ed eventi territoriali non presenti nei database locali. Sfrutta l'integrazione nativa con Tavily Search, un motore di ricerca ottimizzato per agenti AI che restituisce un JSON pulito contenente gli URL sorgente e gli snippet di testo estratti dalle pagine web.
- **source_credibility_tool**: È un modulo euristico-analitico progettato per valutare la qualità algoritmica di una fonte web o di un testo recuperato da Internet, assegnando un punteggio numerico di affidabilità espresso in centesimi (0-100).

  Il tool imposta un ancoraggio neutrale a 70 punti e applica modificatori matematici rigidi analizzando il testo in minuscolo (.lower()):
  - **White-list (+15 punti)***:* Riconosce domini istituzionali, accademici o testate storiche di settore audio (es. soundonsound, audiofader, conservatorio, allmusic).
  - **Sanzioni Clickbait (-25 punti)***:* Rileva l'uso di pattern linguistici sensazionalistici o manipolatori (es. "incredibile", "segreto", "scioccante", "non crederai").
  - **Sanzioni Commerciali (-20 punti):** Rileva stringhe puramente finalizzate alla vendita (es. "compra ora", "sconto immediato", "biglietti limitati").
  - **Superficialità (-10 punti):** Rileva la mancanza di termini tecnici o date.

  L'output viene iniettato nel testo e post-processato dal Writer tramite espressioni regolari per marchiare esplicitamente ogni citazione (es. [Credibilità: 85/100]).

- **web_content_moderator_tool**: È lo strumento cardine basato sul modello RoBERTa Full Fine-Tuned (caricato localmente dalla directory ./fine-tuning/roberta_full tramite pipeline di Hugging Face). Riceve il testo grezzo e rumoroso generato dall'osservazione di Tavily e lo analizza blocco per blocco per classificarlo come divulgativo (approvato) o pubblicitario (spam/gossip da scartare).
    - NOTA: Inizialmente, si verificava un interruzione del workflow per oltre 5 minuti causata dal tool. Infatti, il motore Tavily estrae dal web testo grezzo non strutturato di dimensioni massive (spesso superiori a 10.000 caratteri). Alimentando il modello di classificazione locale RoBERTa direttamente su architettura hardware CPU, la natura quadratica dei Transformer rispetto alla lunghezza della stringa in ingresso generava un sovraccarico computazionale imprevisto. Dunque, all'interno del "web_content_moderator_tool.py" è stata introdotta un'operazione di frammentazione a [:600] caratteri per singolo blocco. L'inferenza sulla CPU è passata istantaneamente da minuti a frazioni di secondo.

NOTA: può capitare che il modello formuli una query che conduce a pagine web non inerenti con il topic. In tal caso, il modello è bravo a capirlo (formulando un thought derivante dall' observation), e, di conseguenza, scarta la ricerca e riformula un'altra query. Ovviamente il link della pagina web non corretta non verrà analizzato dal source credibility tool.

6. Fine-tuning

6.1.Definizione del problema

Il nostro sistema Agentico prevede, tramite il tool "web_content_moderator_tool", che le fonti provenienti dai vari strumenti a disposizione del researcher ("web_search_tool") non siano di natura personale, cioè non prevedano giudizi personali, né di natura pubblicitaria. Per fare ciò vogliamo utilizzare un modello fine-tuned proprio per filtrare questi tipi di fonti e mantenere le fonti di tipo divulgativo.

Il downstream task del modello rientra quindi nella categoria dei Text Classification, nello specifico di Intent Classification; cioè deve capire, attraverso la semantica del testo, il lessico ed il tono, l'intenzione del testo in input. Ad esempio:

"Gentile professionista, siamo lieti di presentarle il nostro nuovo programma di gestione patrimoniale integrata per le piccole e medie imprese. Il sistema consente di ottimizzare i flussi di cassa riducendo l'impatto fiscale delle transazioni transfrontaliere attraverso investimenti mirati in fondi azionari a basso rischio. Vi invitiamo a consultare la nostra brochure informativa per scoprire tutti i dettagli commerciali."

deve essere classificato nella classe "**Pubblicitario**", mentre un testo del tipo:

"Larmonium, o harmonium, è un tipo di organo a serbatoio d'aria con una (o più) tastiere e due pedali per azionare i mantici per l'aria. Il suono è prodotto da una o più file di ance libere, analogamente alla fisarmonica e all'armonica a bocca."

dovrà essere classificato come “**Divulgativo**”.

6.2. Dataset utilizzato

Il dataset utilizzato per il fine tuning è stato ricavato da più dataset provenienti dalla piattaforma HuggingFace:

- Dataset della classe “Divulgativo”: sono stati caricati 1000 record dal dataset “wikimedia/wikipedia" subset: "20231101.it" , che raccoglie testi divulgativi dal sito wikipedia. Il dataset è stato modificato scartando le colonne che non servivano per l’esecuzione del task lasciando due colonne: “text” e “label”.
- Dataset della classe “Pubblicitario”: Per costituire questa classe sono stati caricati 1000 campioni da due dataset diversi. I primi 500 record dal dataset “MelmaGrigia/italian-text-sentiment-analysis”, che contiene frasi progettate per la classificazione delle emozioni, filtrate per le emozioni rabbia, sorpresa e gioia che più simulano il contesto personale. I restanti 500 record sono stati caricati dal dataset “tanaos/synthetic-spam-detection-dataset-italian”, che fornisce testi pubblicitari o di spam.

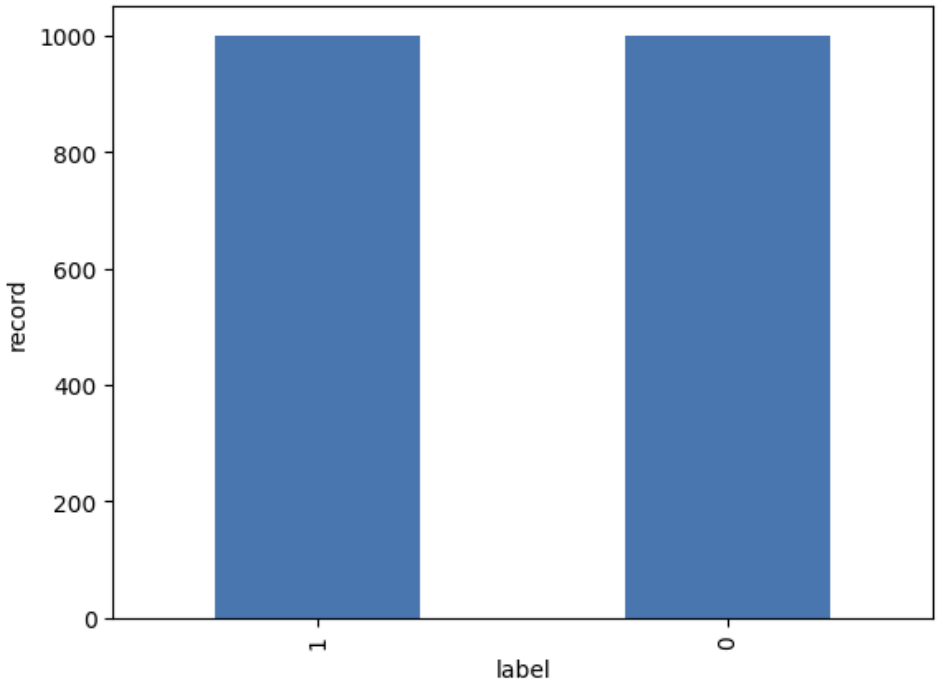
6.2.1. Risoluzione dei bias

Dallo studio e valutazione del dataset della classe “Pubblicitario”, costituito dai due dataset, ci si è resi conto che presentava dei bias lessicali e strutturali in quanto il vocabolario utilizzato dai due dataset (“italian-text-sentiment-analysis” e “synthetic-spam-detection-dataset-italian”) era privo di termini tecnici, scientifici, di cui il dataset della classe “Divulgativo” era fortemente ricco; inoltre, il testo della maggior parte delle istanze pubblicitarie era una frase, mentre il primo dataset contiene paragrafi.

Queste sono le motivazioni che hanno portato a modificare il dataset della classe “Divulgativo” e “Pubblicitario”.

- Riduzione del testo delle istanze del dataset “Divulgativo”: un testo completo di una singola era formato mediamente da otto paragrafi, ciascuno composto da diverse frasi, per cui è stato selezionato, grazie alla funzione *“applica_taglio”*, solamente il primo paragrafo per ogni istanza.
- Espansione del dataset “Pubblicitario”: per superare il bias strutturale sulla lunghezza dei testi e lessicale del dataset di classe 1 sono state generate sinteticamente 404 istanze con testo più lungo e ricco di termini tecnici e scientifici.
- Riduzione delle istanze dal dataset *“italian-text-sentiment-analysis”* da 500 a 298.
- Riduzione delle istanze dal dataset *“synthetic-spam-detection-dataset-italian”* da 500 a 298.

In questo modo abbiamo bilanciato il dataset “Pubblicitario” con 298+298+404 istanze dai 3 diversi dataset, risolto il bias lessicale ed il bias strutturale sulla lunghezza differente dei testi.



Label 1: Classe Pubblicitario Label 0: Classe Divulgativo

Il dataset è stato splittato in 64% train set, 20% test set, 16% validation set.

```
DatasetDict({
    train: Dataset({
        features: ['text', 'label'],
        num_rows: 1280
    })
    test: Dataset({
        features: ['text', 'label'],
        num_rows: 400
    })
    validation: Dataset({
        features: ['text', 'label'],
        num_rows: 320
    })
})
```

Il dataset è stato strutturato quindi in [text;label]
{'text': 'Guadagna 500€ al mese da casa! Il nostro schema ti permetterà di arricchirsi velocemente. Clicca qui: http://gainmoney.site', 'label': 1}

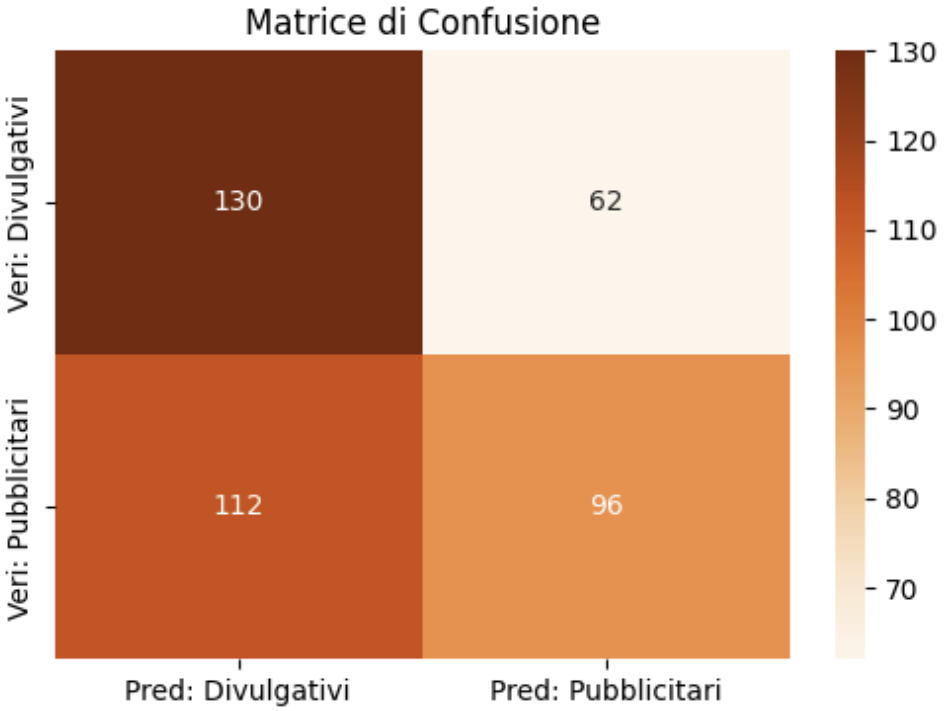
6.3. Metriche di valutazione

Come metrica di valutazione principale del modello, essendo il dataset bilanciato, utilizziamo l' Accuracy. Oltre all'accuratezza andiamo ad osservare anche l' F1 score, in quanto, se il modello ottenesse un' ottima classificazione su una sola della classe, l' F1 calerebbe evidenziando un problema che l'accuratezza potrebbe mascherare.

6.4. Baseline

Come baseline di riferimento per il nostro task abbiamo scelto il modello “xlm-roberta-large-xnli” utilizzato in modalità Zero-shot.

L’ inferenza eseguita sul test set ha restituito 180 errori di classificazione su 400.



Risultati baseline Zero-Shot:

Accuracy: 55%

	Divulgativo	**Pubblicitario**
F1-Score	59%	50%

6.5. Scelta del modello

L' obbiettivo del nostro task è comprendere e classificare testi in lingua italiana, un compito che richiede quindi comprensione del contenuto semantico e dell'intento comunicativo di quest'ultimo. In linea con le linee guida per la selezione dei modelli presenti su (Alammar, 2024)[1] , è stato scelto di utilizzare **osiria/roberta-base-italian**, un modello basato sull' architettura RoBERTa encoder-only. I modelli, come BERT o RoBERTa, encoder-only sono specializzati proprio nell' apprendere informazioni dall'intero input e roberta-base-italian mantiene tutte le caratteristiche di RoBERTa Base con 125 milioni di parametri.

6.6. Prima strategia di Fine-Tuning

La letteratura sulle tecniche di fine-tuning dei foundation model evidenzia come, per adattare un modello già pre-addestrato ad un downstream task, non è necessario riaddestrare tutti i pesi del modello per ottenere una buona accuratezza.

Sulla base di questo paradigma, come primo tentativo di fine-tuning è stata scelta una tecnica della famiglia **PEFT** (Parameter-Efficient Fine-tuning), nello specifico la tecnica **LoRA** (Low Rank Adaptation). Questa strategia impone al modello di costruire una matrice d' aggiornamento dei pesi, ΔW, moltiplicando due matrici A e B di dimensione molto più piccole della matrice originale dei pesi.

$$W_{nuovo} = W_0 + \Delta W$$

$$\Delta W = B \times A$$

Supponendo che la matrice W_0 abbia dimensione $d \times k$, definiamo $r \ll \min(d, k)$.

Il modello costruisce ΔW tramite:

- La matrice A di dimensione $d \times r$
- La matrice B di dimensione $r \times k$

La difficoltà risiede quindi nell'addestrare A e B di rango molto minore rispetto alla dimensione di W_0. Quando il modello andrà a calcolare $B \times A$ la matrice risultante avrà dimensione $d \times k$ combaciando perfettamente con W_0.

LoRA permette di addestrare 1.181.954 parametri su 124 milioni (0.93%).

[1] Jay Alammar et al., Hand-On Large Language Models, 2024

6.6.1. Ricerca Iper Parametri per LoRA

Al fine di identificare gli iperparametri migliori per il fine-tuning è stata svolta una grid search di quattro configurazioni variando il LoRA Rank (r), LoRA Alpha(α) e Learning Rate (lr):

Configurazione	**Rango (r)**	**Alpha (α)**	**Learning Rate (lr)**
Config_1	8	16	$1 \cdot 10^{-4}$
Config_2	16	32	$1 \cdot 10^{-4}$
Config_3	8	16	$2 \cdot 10^{-4}$
Config_4	16	32	$5 \cdot 10^{-5}$

- **Config_1**: (r=8, α=16, lr=1e-4) configurazione standard di fine-tuning per l'addestramento dei parametri con il minimo del calcolo computazionale.
- **Config_2**: (r=16, α=32, lr=1e-4) a parità di learning rate proviamo ad addestrare il doppio dei parametri addestrabili e catturare quindi più pattern possibili.
- **Config_3**: (r=8, α=16, lr=2e-4) raddoppiamo il tasso di apprendimento con un rango più basso del precedente per testare la regolarità della funzione di loss.
- **Config_4:** (r=16, α=32, lr=5e-5) accoppiamo la massima capacità di addestramento dei pesi con un learning rate più basso quindi più granulare.

Ogni configurazione di addestramento è composta da due epoche.

I risultati di ogni configurazione sono i seguenti:

Config_1:

Epoch	**Training Loss**	**Validation Loss**	**Accuracy**	**F1**
1	1.354942	1.266029	0.628125	0.617636
2	1.213398	1.234853	0.696875	0.696730

Config_2:

Epoch	**Training Loss**	**Validation Loss**	**Accuracy**	**F1**
1	1.353209	1.244817	0.640625	0.631259
2	1.185549	1.291933	0.706250	0.705963

Config_3:

Epoch	**Training Loss**	**Validation Loss**	**Accuracy**	**F1**
1	1.365645	1.220094	0.643750	0.636233
2	1.173821	1.245540	0.706250	0.706204

Config_4:

Epoch	Training Loss	Validation Loss	Accuracy	F1
1	1.376745	1.356369	0.668750	0.668115
2	1.315875	1.313851	0.650000	0.639741

La configurazione migliore è risultata la **Config_2** (lr=1e-4 e r=8) con un' accuracy del 70%.

Abbiamo scelto di mantenere costante il dropout, che azzera casualmente una percentuale di neuroni per evitare l' overfitting, a 0.1 per mantenere il minimo rumore che evita al modello di memorizzare il dataset ma non distruggere le capacità apprese.

6.6.2.Fine-tuning con LoRA

Trovata la configurazione degli iper-parametri migliori per LoRA abbiamo svolto il fine-tuning con tre epoche al modello:

Epoch	Training Loss	Validation Loss	Accuracy	F1
1	1.297513	1.167152	0.706250	0.706239
2	1.139595	1.203028	0.718750	0.718750
3	1.095903	1.147009	0.718750	0.718651

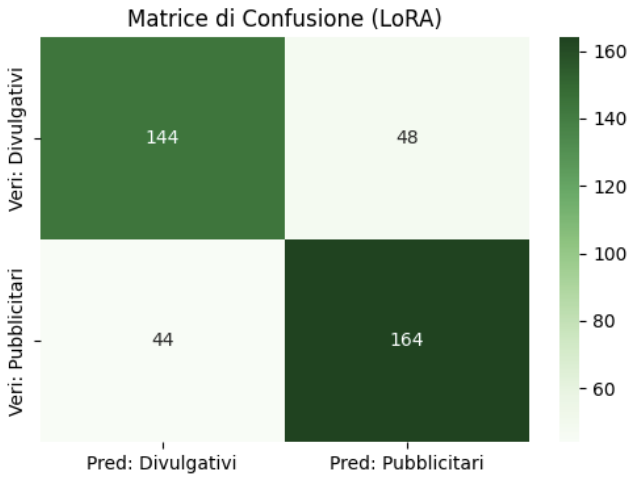
6.6.3.Testing con LoRA

Una volta concluso il fine-tuning del modello è stata svolta l' inferenza sul test set, producendo i seguenti risultati:

Accuracy: 77%

	Divulgativo	Pubblicitario
F1-Score	76%	78%

Producendo la seguente matrice di confusione:



6.6.4.Error Analysis

Al termine della fase di test sono stati analizzati gli errori che ha commesso il modello.

Per esempio:

Testo: *Saper parlare in pubblico in modo chiaro ed efficace è una dote fondamentale per manager e professionisti. Il nostro seminario intensivo di public speaking offre esercitazioni pratiche e tecniche di respirazione per superare l'ansia da prestazione e catturare l'attenzione di qualsiasi tipologia di pubblico fin dalle prime battute.*

Etichetta Vera: pubblicitario

Etichetta Predetta: divulgativo

Testo: *La tua sim card telefonica scadrà e il tuo numero verrà disattivato in via definitiva se non effettui immediatamente una ricarica di importo pari a cinque euro tramite la pagina protetta qui allegata, a causa di nuovi regolamenti ministeriali sulla portabilità delle linee mobili non attive.*

Etichetta Vera: pubblicitario

Etichetta Predetta: divulgativo

Analizzando i diversi errori si può ipotizzare che il modello abbia erroneamente predetto "divulgativo" invece di "pubblicitario" perché il testo, nell'esempio sopra, presenta prima una frase che può sembrare di tipo divulgativo, ma subito dopo, una frase pubblicitaria.

In aggiunta a ciò, il testo dell' istanza presenta un lessico professionale che può ingannare il modello.

Di fronte a testi ambigui, come abbiamo visto, il modello non si comporta come vorremmo, per questo motivo abbiamo scelto di utilizzare come strategia di fine-tuning il **Full fine-tuning**.

6.7. Seconda strategia di Fine-tuning

Visto come la strategia PEFT non ha portato i risultati sperati, abbiamo scelto di utilizzare la strategia del **Full fine-tuning**.

Considerate le dimensioni non eccessive del modello, 124M di parametri, il full fine-tuning ci permette, in tempi moderati, di addestrare l'intero spazio parametrico di RoBERTa. Questo consente al modello la massima flessibilità per capire anche i pattern linguistici più complessi, come, nel nostro caso, riuscire a capire se si tratta di un testo pubblicitario nonostante utilizzi un linguaggio tecnico.

6.7.1. Scelta degli iperparametri per il Full fine-tuning

L'obbiettivo del full fine-tuning non è re-inizializzare il modello, ma adattare i pesi già pre-addestrati in fase di pre-training al nostro specifico downtask. Il modello possiede già una conoscenza acquisita in fase di pre-training, per non rischiare la perdita di queste conoscenze, **Catastrophic Forgetting**, impieghiamo un learning rate di 10^{-5} per garantire un' ampiezza sufficientemente ridotta.

6.7.2. Fine-tuning con FFT

La strategia di Full fine-tuning con learning rate 10^{-5} in tre epoche ha restituito le seguenti statistiche:

Epoch	Training Loss	Validation Loss	Accuracy	F1
1	1.265899	1.126506	0.703125	0.701583
2	0.992838	0.839802	0.765625	0.764962
3	0.883512	0.781111	0.784375	0.783612

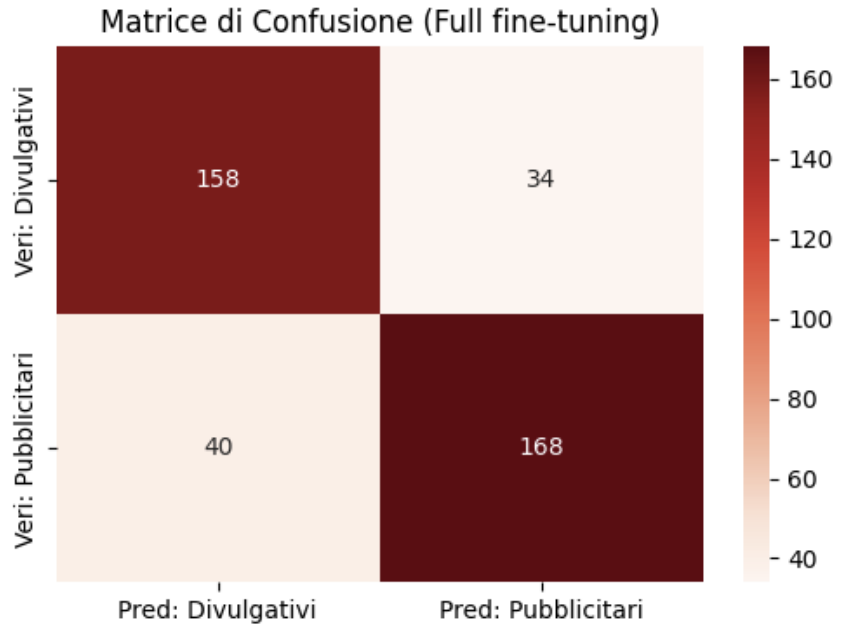
6.7.3. Testing con FTT

Conclusa la fase di training con il Full fine tuning è stata svolta l'inferenza del modello sul test set, producendo i seguenti risultati:

Accuracy: 81%

	Divulgativo	Pubblicitario
F1-Score	81%	82%

Producendo la seguente matrice di confusione:



6.8. Confronto con Baseline

Dopo aver eseguito diverse strategie di fine-tuning, confrontiamo ora le prestazioni del modello con la baseline iniziale:

Modello	F1-Divulgativo	F1-Pubblicitario	Accuracy
xlm-roberta-large-xnli (Zero-Shot)	59%	50%	55%
osiria/roberta-base-italian (LoRA)	76%	78%	77%
osiria/roberta-base-italian (FFT)	81%	82%	81%

Il modello **xlm-roberta-large-xnli** utilizzato in modalità zero-shot registra le performance peggiori, poco superiore ad una classificazione casuale, dimostrando che un modello, anche se di grandi dimensioni e pre-addestrato sulla lingua italiana con 30Gb di testo italiano[2], non possiede le capacità per il nostro task di classificazione.

Passando al modello **osiria/roberta-base-italian** addestrato tramite **LoRa** si ha un incremento delle prestazioni di Accuracy del +22% con un' aggiornamento dei parametri originali minore dell' 1%, dimostrando la tesi che con un minimo costo computazionale otteniamo un ottimo risultato.

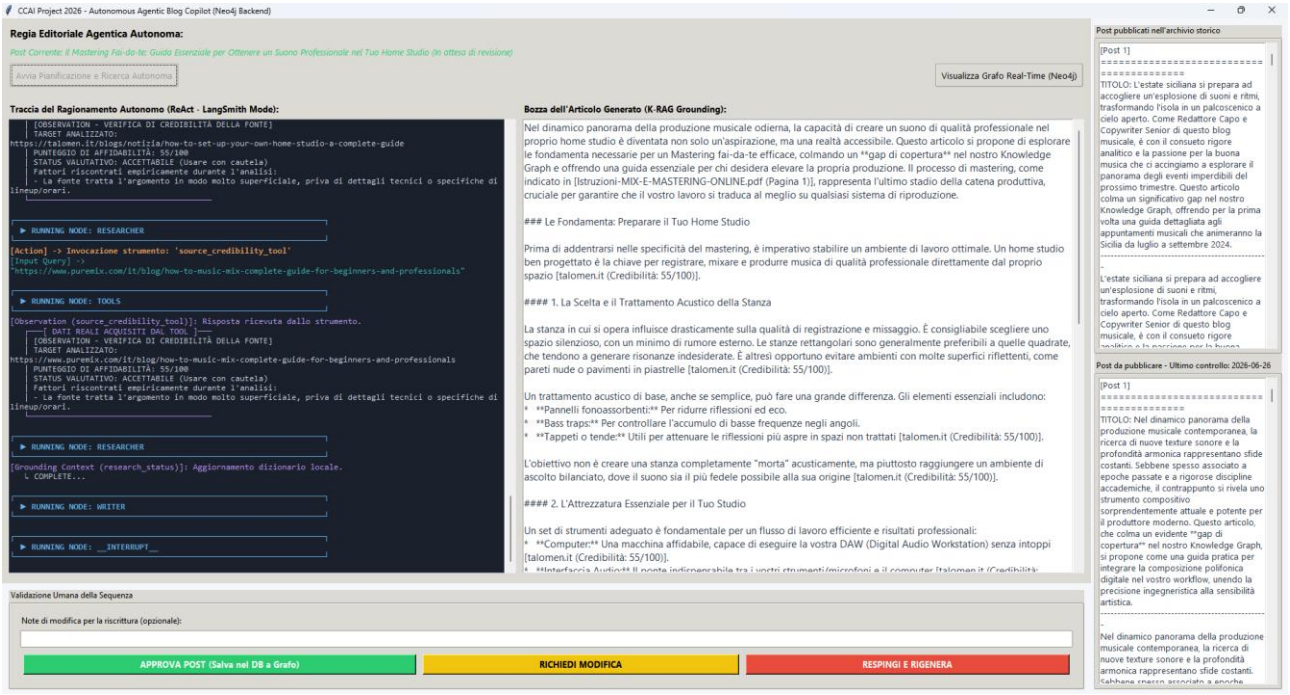
Sbloccando l'intero spazio parametrico del modello con il **Full fine-tuning** riusciamo a raggiungere l' 81% di Accuracy. Inoltre, la vicinanza tra i due punteggi di F1 dimostra che il modello è riuscito a discriminare le due classi equamente.

[2] [Unsupervised Cross-lingual Representation Learning at Scale](https://aclanthology.org/2020.acl-main.747/) (Conneau et al., ACL 2020)

7. Valutazione

L'architettura è stata adattata per supportare due diverse modalità di esecuzione del sistema:

- Utilizzo della piattaforma LangSmith, utile per visualizzare come variano i parametri di osservabilità (come token consumati e latenza di esecuzione del nodo e dell'LLM), e i campi dello stato. L'utilizzo di LangSmith è stato determinante per effettuare il debugging e identificare i Failure Case.
- È stata realizzata una GUI (all'interno dello script "main.py"), implementata tramite la libreria Tkinter, in modo da fornire all'utente una modalità di esecuzione più rapida. Quest'interfaccia permette di visualizzare in modo generale le risposte del planner, la reasoning trace del researcher e gli output dei vari tool, oltre al post finale. L'interazione con l'utente (Human-in-the-loop) è favorita dall'implementazione di tre pulsanti, oltre che un input testuale in cui inserire delle note di modifica.



Tuttavia, il team ha preferito effettuare una maggiore percentuale di test sulla piattaforma LangSmith, proprio per i motivi appena elencati. La GUI rimane comunque una modalità veloce, semplice ed efficiente di esecuzione per utenti esterni che si interfacciano con l'applicazione.

Questa fase di verifica ha permesso di misurare l'accuratezza del grounding, l'efficacia della moderazione locale e la sostenibilità computazionale del grafo rispetto ai consumi di token e alle latenze di risposta.

7.1. Analisi Qualitativa dei Post Generati

Prendendo come esempio un determinato test effettuato su LangSmith, l'analisi qualitativa dei contenuti generati esamina l'output del nodo Writer per un topic strategico assegnato dal Planner: *"Jimi Hendrix: L'Innovazione Chitarristica, il Blues Elettrico e l'Eredità di un Icona Rock"*.

L'elaborato prodotto dimostra un'elevata aderenza ai vincoli stilistici e strutturali imposti:

- **Pertinenza e Prosa Editoriale:** Il modello (Gemini 2.5 Flash) ha declinato correttamente la categoria Product review, fondendo la terminologia tecnica dell'ingegneria acustica con la fluidità della critica musicale.
- **Organizzazione Strutturale:** Il testo rispetta la tripartizione tematica definita in fase di pianificazione, articolando l'analisi in sezioni logiche mediante marcatori Markdown (###) dedicate all'ingegneria del suono, alla fusione dei generi e all'eredità storico-culturale dell'artista.

7.2. Valutazione della Qualità delle Fonti e del Grounding (Fondatezza)

Il sistema esibisce un comportamento deterministico e privo di allucinazioni, ancorando ogni asserzione tecnica o biografica ai soli dati convalidati nel flusso di ricerca.

La fondatezza dell'articolo è garantita dall'unificazione di canali informativi eterogenei:

- **Knowledge Graph:** L'agente ha cercato correttamente il topic all'interno di Neo4j. E' stata trovata una correlazione con il topic *"Pink Floyd: L'Architettura Sonora e il Concetto nell'Evoluzione del Rock Progressivo"*. Chiaramente il topic non è del tutto inerente al topic corrente, altrimenti non si tratterebbe di un gap. Tuttavia, estraendo il blocco sui Pink Floyd, il sistema acquisisce il contesto editoriale del blog (lo stile, l'uso del grounding e i parametri di recensione) per mantenere la coerenza stilistica. Infatti, a tal proposito, nell'introduzione del post, il writer utilizza il post precedente come rampa di lancio (*"Se abbiamo esplorato come i Pink Floyd abbiano costruito un'architettura sonora e concettuale complessa, in questo 'product review' ci rivolgiamo..."*). Inoltre, collega l'ingegneria del suono di Hendrix alla *"meticolosa attenzione alla struttura e al dettaglio sonoro"* dei Pink Floyd. Infine, riprende il concetto di *"perfetta calibrazione"* sonora (estratto esattamente dal testo d'archivio dei Pink Floyd) per unificare l'eredità dei due pilastri del rock.
    - NOTA: Volendo un approccio più restrittivo sulla selezione dei topic, si potrebbe decidere di alzare la soglia
- **RAG Vettoriale Locale:** Il sistema estrae 2 chunk dal documento Freddie Mercury.pdf (entrambi Pag. 12), inserendo la citazione testuale esatta in cui Freddie Mercury definisce Hendrix il proprio idolo. Nonostante anche in questo caso, non vi è una pura correlazione con il topic attuale, il writer riesce comunque ad inserire questi contenuti all'interno dell'articolo generato, affermando come Jimi Hendrix fosse un idolo per Freddie mercury, citando il nome del file.pdf, oltre alla pagina, e attribuendo al post un maggior grado di completezza.
- **Ricerca Web Esterna:** I dettagli sull'artista e sulle tecniche esecutive vengono ereditati fedelmente dallo snippet di Wikipedia. Infatti, tutti i dettagli tecnici inseriti, come la data di nascita e morte (27 novembre 1942 – 18 settembre 1970), le tecniche (wah wah, feedback, distorsioni, Larsen, dissonanze, Double stop, legati, cromatismi), l'uso delle scale pentatoniche, l'influenza su Miles Davis in Bitches Brew, il contributo all'heavy metal e la definizione dell'Hendrix Chord (nona diesis),

derivano direttamente dallo snippet enciclopedico, senza alcuna estensione arbitraria.

Inoltre, il sistema ha scansionato i link in uscita e ha iniettato i punteggi di affidabilità calcolati dal source_credibility_tool, discriminando l'autorevolezza enciclopedica di Wikipedia ([Credibilità: 80/100]) rispetto alla superficialità informativa di Instagram ([Credibilità: 55/100]). Per i documenti d'archivio e il RAG interno, è stata applicata la stringa di protezione [Credibilità: Verificata].

7.3. Metriche di Osservabilità

Attraverso LangSmith, è stata effettuata la mappatura temporale e volumetrica dei singoli nodi del grafo. I dati sono relativi alla stesura di un singolo post (lo stesso esaminato nei paragrafi precedenti)

Nodo / Tool	**Latenza Totale (s)**	**Latenza LLM (s)**	**Consumo di Token di input (prompt) e di output (completion)**
planner	30.62 s	25.67 s	9.1 K
researcher (Turno 1)	3.89 s	2.54 s	1.9 K
query_knowledge_graph_tool	5.46 s	-	-
researcher (Turno 2 - post KG)	2.95 s	1.79 s	6.3 K
rag_retrieval_tool	8.60 s	-	-
researcher (Turno 3 - post RAG)	4.06 s	2.73 s	7.1 K
web_search_tool (Tavily)	3.95 s	-	-
researcher (Turno 4 - post Web)	5.62 s	4.53 s	8.8 K
web_content_moderator_tool	2.57 s	-	-

Nodo / Tool	Latenza Totale (s)	Latenza LLM (s)	Consumo di Token di input (prompt) e di output (completion)
researcher (Turno 5 - post Mod)	4.26 s	3.31 s	9.4 K
source_credibility_tool	0.00 s	-	-
researcher (Turno Finale)	3.08 s	1.90 s	9.9 K
writer	15.08 s	13.76 s	17.0 K
update_knowledge_graph	5.63 s	4.43 s	2.5 K

Dai dati appena riportati, emergono le seguenti considerazioni:

1. Il picco volumetrico si registra sul nodo writer con **17K token**. Considerando che Gemini 2.5 Flash supporta un milione di token, il sistema impegna una bassa percentuale della capacità massima, garantendo costi di computazione infinitesimali.
2. I **15.08s** richiesti dal writer sono interamente dominati dal tempo di compilazione dei token in uscita necessari alla stesura dell'articolo esteso. I **30.62s** del planner riflettono una latenza iniziale di rete legata alla ricerca dei topic nel database Neo4J e all'analisi di essi. Nel complesso, il sistema impiega circa un minuto e mezzo per la scrittura di un articolo.
3. Le metriche mostrano che, subito dopo l'approvazione del post, il nodo update_knowledge_graph abbatte il volume dello stato a **2.5K token**. Questo dato dimostra il successo dell'operazione di Garbage Collection Semantica indotta dai comandi RemoveMessage, che svuotano la chat ReAct accumulata azzerando il carico cognitivo per il topic successivo della roadmap.

7.4. Identificazione dei Casi di Fallimento

I vari test condotti hanno permesso di mappare tre specifici casi di fallimento, dovuti alla deriva comportamentale dei modelli.

1. **Caso di Fallimento 1: Mancanza di inserimento di Thought e Action nel Researcher**

Durante i turni avanzati del ciclo ReAct, In alcuni casi l'agente Researcher omette la generazione del token discorsivo THOUGHT, saltando direttamente alla compilazione del blocco JSON di chiamata del tool (ACTION). Talvolta vengono entrambi omessi, e si procede direttamente all'invocazione del tool. Nonostante è stata inserita, nel RESEARCHER_SYSTEM_PROMPT, una regola di Prompt Engineering ("Esplicita sempre THOUGHT ed ACTION prima di chiamare un tool"), l'LLM è un motore probabilistico, non un programma deterministico. Quando la chat ReAct si allunga e lo Stato accumula molti messaggi (specialmente dopo che Tavily ha rovesciato nei log snippet complessi, link di YouTube e tag HTML), il modello sperimenta il cosidetto "Attentional Drift" (Deriva dell'Attenzione) o perdita di aderenza alle istruzioni di sistema. L'LLM si focalizza così tanto sui dati tecnici estratti che "dimentica" la formattazione testuale dei pensieri e invoca direttamente la tool_call di cui ha bisogno.

2. **Caso di Fallimento 2: Omissione del Titolo dell'articolo**
   Alcune volte, Il Writer omette la generazione del titolo principale, causando un fallimento a catena nel nodo update_kg_node, il quale estrae la prima riga di testo come metadato identificativo del post. Ciò avviene perchè L'LLM interpreta la chiave esplicita Topic: ... presente nell'input strutturato come un elemento di intestazione già acquisito dal sistema, avviando la stesura direttamente dalle sezioni secondarie.
3. **Caso di Fallimento 3: Deriva Semantica nella Generazione delle Query (Query Drift)**
   Può capitare in alcuni casi che l'LLM partendo da un topic chiaro, formula una query di ricerca per Tavily troppo generica o fuorviante. Questo porta il motore di ricerca a restituire pagine web non pertinenti. Ciò è dovuto ad un fenomeno di allineamento difettoso del contesto (Context Misalignment). Durante il ciclo ReAct, l'LLM può farsi "condizionare" da una singola parola trovata nei passaggi precedenti (magari un nodo del Knowledge Graph o un chunk del RAG) e perdere il focus sull'obiettivo principale stabilito dal Planner. Di conseguenza, genera una stringa di ricerca che devia semanticamente verso la cronaca o il gossip, inquinando i risultati della ricerca web. Tuttavia, nel ciclo successivo, il modello è bravo a capirlo, scarta la ricerca e formula un'altra query. Infatti, il source_credibility_tool non viene invocato in quella fonte.